

BookNest

Smart Library & E-Commerce Portal

Report submitted as part of the internship program requirement for the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING (AI & ML)

Submitted by:

NARLA SAMHAVYA SUBHANVITHA (A24126552099)



DEPARTMENT OF CSE (AI & ML)

**ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES
(UGC AUTONOMOUS)**

**(Permanently Affiliated to AU, Approved by AICTE, and Accredited by NBA &
NAAC)**

**SANGIVALASA, BHEEMILI MANDAL, VISAKHAPATNAM – 531162
2024–2028**

**ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Affiliated to Andhra University)**

SANGIVALASA, VISAKHAPATNAM – 531162

2024–2028



BONAFIDE CERTIFICATE

This is to certify that this Case Study Report titled “AUTOMATED FINANCIAL CREDIT & LOAN DECISION ENGINE” from IBM is the bonafide work of NARLA SAMHAVYA SUBHANVITHA (A24126552099) of III/IV CSE (AI & ML), carried out under the guidance and supervision of the Department of CSE (AI & ML), Anil Neerukonda Institute of Technology and Sciences.

REVIEWER

Mrs Moni Sushma Deep
Assistant Professor
Department of CSE
(AI & ML)
ANITS

CLASS TEACHER

P Santosh Kumar
Assistant Professor
Department of CSE
(AI&ML)
ANITS

HEAD OF THE DEPARTMENT

DR. K. SELVANI DEEPTHI
Department of CSE (AI & ML)

ANITS

ACKNOWLEDGEMENT

The successful completion of a project is the result of continuous learning, guidance, encouragement, and support from many individuals. I take this opportunity to express my sincere gratitude to everyone who contributed to the successful completion of this case study project.

First and foremost, I express my sincere gratitude to Dr. K. S. Deepthi, Head of the Department of Computer Science & Engineering (AI & ML), ANITS, for providing the necessary support, guidance, and encouragement throughout the development of this project.

I would also like to express my heartfelt thanks to my faculty members and project mentors for their valuable suggestions, technical guidance, and continuous encouragement during the different stages of the project. Their feedback helped me improve my understanding of machine learning and its practical applications in financial decision-making.

I am grateful to the Department of CSE (AI & ML), Anil Neerukonda Institute of Technology and Sciences, for providing the academic environment and resources required to carry out this project successfully.

I would also like to thank my classmates and friends for their support, discussions, and suggestions during the development and testing of the project.

Finally, I express my sincere gratitude to everyone who directly or indirectly contributed to the successful completion of the “BOOKNEST” project.

NARLA SAMHAVYA SUBHANVITHA

(A24126552099)

Table of Contents

1. Introduction

- 1.1 Background of the Internship / Project
- 1.2 Objectives of the Internship
- 1.3 Importance of Full Stack Web Development Skills
- 1.4 Scope of the Project

2. Organization Profile

- 2.1 About Edumoon
- 2.2 Program Structure and Mentorship Model

3. Project Overview

- 3.1 Title of the Project
- 3.2 Problem Statement
- 3.3 Objectives of the Project
- 3.4 Relevance and Real-World Use Case

4. Literature Review / Theoretical Background

- 4.1 Version Control and Collaborative Development
- 4.2 Front-End Fundamentals (HTML, CSS, JavaScript)
- 4.3 Back-End Development with Node.js and Express
- 4.4 Databases: SQL, NoSQL and MongoDB
- 4.5 Authentication, OTP Verification and Security
- 4.6 Machine Learning for Spam Detection
- 4.7 REST API Design and Client-Server Architecture
- 4.8 Hosting and Deployment (Netlify, Render)

5. Methodology / Development Roadmap

- 5.1 Phase 1 — Front-End Foundations
- 5.2 Phase 2 — Backend API and Database Modelling
- 5.3 Phase 3 — Cart, Wishlist, Orders and Payments
- 5.4 Phase 4 — Author Interaction and Reader Wall
- 5.5 Phase 5 — Machine Learning Integration and Deployment

6. Implementation

- 6.1 Week-by-Week Implementation Schedule
- 6.2 Backend Implementation — BookNest API
- 6.3 Frontend Implementation
- 6.4 Spam Detection Microservice Integration
- 6.5 Admin (Library Management) Module

7. Results and Discussion

- 7.1 Features Delivered
- 7.2 API Endpoint Summary
- 7.3 Application Walkthrough
- 7.4 Security Measures and Current Gaps

8. Conclusion & Future Work

- 8.1 Summary of Learning
- 8.2 Skills Acquired
- 8.3 Limitations of the Current Approach
- 8.4 Future Scope

9. Internship / Project Outcomes

- 9.1 Technical Skills Acquired
- 9.2 Soft Skills Developed
- 9.3 Contribution to Career Growth

10. Source Code and Project Structure

- 10.1 Project Directory Structure
- 10.2 Backend Source Code
- 10.3 Frontend Source Code
- 10.4 Spam Detection Service Source Code

Appendix

- A. Source Code (GitHub Repository Link)
- B. Application Screenshots
- C. Certificate of Completion

1. Introduction

1.1 Background of the Internship

This report presents the design, development and outcomes of BookNest, a full stack web application that combines a library management system with an e-commerce book store. The project was undertaken as part of a Full Stack Web Development internship, in fulfilment of the requirements of the Bachelor of Technology in Computer Science and Engineering (AI & ML).

BookNest was built end-to-end — from database schema design and REST API development to a responsive multi-page front end — and additionally integrates a machine-learning-based spam detection microservice, which was learned and applied independently alongside the core MERN-adjacent stack (MongoDB, Express.js, HTML/CSS/JavaScript and Node.js).

1.2 Objectives of the Internship

- To gain hands-on experience in designing and implementing a REST API using Node.js and Express.
- To model and persist application data using MongoDB Atlas and Mongoose schemas.
- To build a multi-page, responsive front end using HTML, CSS and vanilla JavaScript, without a front-end framework.
- To implement e-commerce style flows: cart, wishlist, checkout and order tracking.
- To implement a secondary verification mechanism (OTP) as part of the authentication flow.
- To explore applied machine learning by building and integrating a spam-classification microservice using scikit-learn and FastAPI.
- To deploy a full stack application using modern hosting platforms (Netlify for the front end, Render for the backend).

1.3 Importance of Full Stack Web Development Skills

Full stack development skills allow an engineer to reason about and contribute to an application end-to-end — from what a user sees and interacts with, to how their data is stored, secured and served. This breadth shortens the feedback loop between an idea and a working product, and enables debugging across the client, network and server rather than being confined to a single layer. Building BookNest across the browser, the Express API and the MongoDB data layer — and additionally wiring in a separate Python-based microservice — also gave practical exposure to how independently deployed services communicate with one another in a real application.

1.4 Scope of the Project

The scope of BookNest was to design and build a multi-page web application that allows a reader to: create an account and sign in; browse and search a catalogue of books; add books to a cart or wishlist;

complete a checkout and view past orders; verify actions using a one-time password (OTP); message a book's author after purchase, subject to an automated spam check on the message; and leave reviews on a public “Reader Wall”. A parallel admin-facing module allows library staff to add, lend, return and remove books from the catalogue, and a dashboard summarizes overall usage (total users, books and orders).

2. Organization Profile

2.1 About Edumoon

Edumoon served as the training and mentorship organization that structured and delivered this Full Stack Web Development internship. Rather than a purely lecture-based format, the program followed a project-based, mentor-guided model — each topic in the 27-part curriculum was paired with hands-on coding, so that concepts such as authentication, database design, or real-time communication were learned by building a working feature rather than by studying them in the abstract.

The program's design reflects a broader trend among modern internship and upskilling providers of pairing structured, sequential technical content with an open-ended capstone project, allowing interns to demonstrate applied competence rather than only completion of exercises.

2.2 Program Structure and Mentorship Model

Aspect	Details
Training Partner	Edumoon
Duration	10-05-2026 to 25-06-2026 (approx. 7 weeks)
Curriculum	27 structured, sequential sessions covering the MERN stack, from Git/GitHub to deployment
Guided Project Module	Real-time chat application (React, MUI, WebSockets, WebRTC)
Capstone Project	Independent full stack web application — BookNest
Mentor	

3. Project Overview

3.1 Title of the Project

BookNest — Smart Library E-Commerce Portal

(Built with Node.js, Express, MongoDB Atlas and vanilla JavaScript, with a Python/FastAPI microservice for spam detection)

3.2 Problem Statement

Independent and college libraries that also wish to sell books commonly need two separate systems: one to manage lending and returns, and another to sell books online. Readers, in turn, want a single place to discover a book, read a preview, buy or borrow it, and discuss it with other readers or the author — without switching between disconnected platforms. BookNest addresses this by combining library-style borrow/return administration with an e-commerce storefront in a single application.

3.3 Objectives of the Project

- Build a account system for readers to sign up and sign in.
- Provide a searchable book catalogue with categories, pricing and availability.
- Implement cart and wishlist functionality, and a checkout flow that records orders and payments.
- Provide an OTP-based verification step for sensitive actions.
- Allow verified purchasers to message a book's author, with messages automatically screened for spam.
- Provide a public “Reader Wall” where verified buyers can share their reading experience for a book.
- Provide an admin/library module to add, lend, return and delete books, plus a usage dashboard.

3.4 Relevance and Real-World Use Case

The project mirrors the shape of production e-commerce and content platforms: a REST API backed by a document database, a client application that consumes that API over HTTP, and a decoupled machine-learning microservice that a service can call for a specific task — the same pattern used when a production application calls out to a recommendation engine, a fraud-detection service, or, as in this case, a spam classifier. The project was deployed as two independently hosted services (a Netlify-hosted front end and a Render-hosted backend), reflecting how modern web applications are actually shipped.

4. Literature Review / Theoretical Background

4.1 Version Control and Collaborative Development

Git is a distributed version control system that tracks changes to source code over time, while GitHub provides remote hosting and collaboration tooling on top of it. The BookNest codebase is organized as a single Git repository with three top-level modules — frontend, backend and spam — allowing each part of the stack to be worked on, tracked and deployed somewhat independently.

4.2 Front-End Fundamentals (HTML, CSS, JavaScript)

HTML provides the semantic structure of each page, CSS governs its visual presentation, and JavaScript adds interactivity by manipulating the Document Object Model (DOM) and communicating

with the backend API using the Fetch API. BookNest deliberately uses vanilla JavaScript rather than a front-end framework: each page (shop, cart, wishlist, orders, profile, and so on) is a standalone HTML file that pulls in a single shared script.js and style.css, and DOM elements are updated directly (e.g. rebuilding the book grid's innerHTML from the API response).

4.3 Back-End Development with Node.js and Express

Node.js is a JavaScript runtime built on Chrome's V8 engine that enables event-driven, non-blocking I/O suited to building network applications such as APIs. Express is a minimal, unopinionated web framework layered on top of Node.js that structures an application around routes and middleware. The BookNest backend is a single Express application (server.js) that registers CORS and JSON body-parsing middleware, defines Mongoose models for every resource, and exposes over 30 REST routes covering users, books, cart, wishlist, orders, payments, OTPs, author messages and the reader wall.

4.4 Databases: SQL, NoSQL and MongoDB

Relational (SQL) databases organize data into fixed-schema tables with relationships enforced by keys, while non-relational (NoSQL) databases favor flexible, document-based schemas that are well suited to rapidly evolving application data. BookNest uses MongoDB Atlas, a cloud-hosted, document-oriented NoSQL database, accessed through Mongoose — an Object Data Modeling (ODM) library that provides schema definition and query building on top of MongoDB's native driver. Ten Mongoose models are defined: User, Book, Cart, Wishlist, Order, Otp, Payment, ReaderWall, Message and Author.

4.5 Authentication, OTP Verification and Security

The current implementation authenticates users by comparing the submitted email and password directly against the stored User document, and issues a One-Time Password (OTP) — a short-lived numeric code, valid for 300 seconds via a MongoDB TTL-style expires field — as a secondary verification step for sensitive actions such as checkout. This is a functional, first-pass authentication scheme; Section 7.4 and Section 8.3 discuss, transparently, where this differs from production-grade practice (e.g. password hashing and token-based sessions) and how it could be hardened.

4.6 Machine Learning for Spam Detection

Beyond the core stack, BookNest integrates a supervised machine learning model to filter spam from author messages and reviews. A text classifier is trained on a labelled SMS/message spam dataset (spam.csv) using scikit-learn, serialized with pickle (spam_model.pkl and vectorizer.pkl), and served behind a small FastAPI microservice (spam/app.py). Incoming messages are first checked against fast rule-based heuristics — embedded URLs, 10-digit phone numbers, and known spam keywords — and only messages that pass those checks are vectorized and classified by the trained model, combining the precision of hand-written rules with the generalization of a learned classifier.

4.7 REST API Design and Client-Server Architecture

REST (Representational State Transfer) structures an API around resources (books, orders, cart items, and so on) manipulated through standard HTTP verbs — GET, POST, PUT/DELETE. BookNest's frontend is a fully decoupled client: every page calls a single constant, API, pointing at the deployed backend URL, and uses the Fetch API to issue JSON requests, mirroring how a mobile app or a separate single-page application would consume the same backend.

4.8 Hosting and Deployment (Netlify, Render)

Netlify provides static-site hosting with continuous deployment from a Git repository, well suited to the plain HTML/CSS/JS front end. Render provides managed Node.js hosting for the Express backend. Deploying the two halves of BookNest as separate services, each with its own URL, reflects a common production pattern where the front end and API evolve and scale independently of one another.

5. Methodology / Development Roadmap

Development of BookNest followed five broad, sequential phases, summarized below.

5.1 Phase 1 — Front-End Foundations

Static page structure and styling for the core pages — index (landing page), signin, signup and shop — followed by a shared style.css design system (navigation bar, card layout, colour palette) applied consistently across every page.

5.2 Phase 2 — Backend API and Database Modelling

Initialization of the Express server, connection to MongoDB Atlas via Mongoose, and definition of the core schemas (User, Book) with the first CRUD routes: signup, signin, and listing books.

5.3 Phase 3 — Cart, Wishlist, Orders and Payments

Addition of the Cart, Wishlist, Order and Payment schemas and their routes, along with the corresponding cart.html, wishlist.html, orders.html and payment.html front-end pages and script.js functions to drive them.

5.4 Phase 4 — Author Interaction and Reader Wall

Addition of the Author, Message, ReaderWall and Experience schemas, the purchase-gated author-messaging route (a user must have an existing order containing the relevant book before a message is accepted), and the reader-wall page for posting and browsing reviews.

5.5 Phase 5 — Machine Learning Integration and Deployment

Training and export of the spam-classification model, building the FastAPI microservice around it, wiring the authorMessage route to a rule-based pre-filter, and finally deploying the front end to Netlify and the backend to Render for public access.

6. Implementation

6.1 Week-by-Week Implementation Schedule

Week	Focus Area
Week 1	Requirement analysis, wireframing, and static front-end pages (index, signin, signup)
Week 2	Express server setup, MongoDB Atlas connection, User and Book schemas, signup/signin routes
Week 3	Shop page, book catalogue rendering, Cart and Wishlist schemas and routes
Week 4	Orders, Payment and OTP schemas and routes; checkout flow end-to-end
Week 5	Author, Message and ReaderWall schemas; purchase-gated author messaging
Week 6	Spam-detection model training and FastAPI microservice; integration with authorMessage
Week 7	Admin (library) module — add/borrow/return/delete books; dashboard route
Week 8	Testing, styling polish, and deployment (Netlify + Render)

6.2 Backend Implementation — BookNest API

The Express application is initialized with CORS and JSON body-parsing middleware, and connects to MongoDB Atlas using a connection string supplied through an environment variable:

server.js — application setup and database connection

```
const express = require("express");
const mongoose = require("mongoose");
const bodyParser = require("body-parser");
const cors = require("cors");
const app = express();
const PORT = 3000;
app.use(cors());
app.use(bodyParser.json());
mongoose.connect(process.env.MONGO_URI)
  .then(()=>{
    console.log("MongoDB Connected");
  })
  .catch((err)=>{
```

```
console.log(err);
});
```

Each resource is represented as a Mongoose schema. For example, the Book schema captures a book's catalogue metadata, availability and rating:

server.js — Book schema

```
const bookSchema = new mongoose.Schema({
  title:String,
  author:String,
  category:String,
  description:String,
  image:String,
  price:Number,
  availableCopies:Number,
  status:String,
  rating:Number
},
{
  timestamps:true
});
const Book = mongoose.model("Book",bookSchema);
```

The Order schema stores the books purchased, total amount and status for each checkout:

server.js — Order schema

```
const orderSchema = new mongoose.Schema({
  userEmail:String,
  books:Array,
  totalAmount:Number,
  status:String,
  orderDate:{
    type:Date,
    default:Date.now
  }
},
{
  timestamps:true
});
const Order = mongoose.model("Order",orderSchema);
```

User registration and login are implemented as straightforward document lookups against the User collection:

server.js — /signup and /signin routes

```
app.post("/signup",async(req,res)=>{
  try{
    const user=await User.findOne({
      email:req.body.email
    });
    if(user){
      return res.send("User already exists");
    }
    const newUser=new User({
      email:req.body.email,
      password:req.body.password
    });
    await newUser.save();
    res.send("Signup successful");
  }
  catch(error){
    res.status(500).send(error);
  }
});
//===== SIGNIN =====
app.post("/signin",async(req,res)=>{
  try{
    const user=await User.findOne({
      email:req.body.email,
      password:req.body.password
    });
    if(!user){
      return res.status(404)
        .send("Account not found");
    }
    res.send("Signin successful");
  }
  catch(error){
    res.status(500).send(error);
  }
});
```

Cart items are persisted per request rather than only in the browser, giving the API routes below:

server.js — /cart/add, /cart and /cart/:id routes

```
app.post("/cart/add",async(req,res)=>{
  const item=new Cart(req.body);
  await item.save();
  res.send("Added to cart");
});
```

```

app.get("/cart", async(req, res) => {
  const cart = await Cart.find();
  res.send(cart);
});
app.delete("/cart/:id", async(req, res) => {
  try {
    await Cart.findByIdAndDelete(req.params.id);
    res.send("Cart item deleted");
  }
  catch(error) {
    res.status(500).send(error);
  }
});

```

6.3 Frontend Implementation

Every front-end page shares a single `script.js` file, which points at the deployed backend through one constant:

script.js — API base URL

```
const API = "https://booknest-backend-nxnk.onrender.com";
```

The shop page fetches the full book catalogue and dynamically builds a card for every book, wiring up Add to Cart, Add to Wishlist, “Message Author” and “Read Preview” actions for each one:

script.js — loadBooks(): fetches and renders the catalogue

```

async function loadBooks() {
  const container = document.getElementById("shopContainer");
  if (!container) return;
  container.innerHTML = "";
  try {
    const res = await fetch(`${API}/books`);
    const books = await res.json();
    console.log(books);
    books.forEach(book => {
      container.innerHTML += `
<div class="book-card">

<h3>${book.title}</h3>
<p>${book.author}</p>
<p>₹${book.price}</p>
<div class="btn-group">
<div class="btn-row">
<button onclick='addToCart(${JSON.stringify(book)})'>

```

```
 Add To Cart
</button>
<button onclick='addToWishlist(${JSON.stringify(book)})'>
 Wishlist
</button>
</div>
<div class="btn-row">
  ${getMessageButton(book)}
  ${getBookButton(book.title)}
</div>
```

The cart itself is kept client-side in the browser's localStorage, so items persist across page loads without an extra network round-trip for every click:

script.js — addToCart(): adds a book to the local cart

```
function addToCart(book) {
  if (!checkLogin()) return;
  let cart = JSON.parse(localStorage.getItem("cart")) || [];
  cart.push(book);
  localStorage.setItem("cart", JSON.stringify(cart));
  alert("Book added to Cart");
}
```

6.4 Spam Detection Microservice Integration

Author messages and reviews are first checked by a lightweight in-Node isSpam() helper (URL, phone-number and keyword heuristics) before being accepted; the same set of checks, plus the trained scikit-learn classifier, is also exposed as a standalone FastAPI service so it can be called independently of the main API or reused by other clients. See Section 10.4 for the full prediction handler.

6.5 Admin (Library Management) Module

A separate set of admin-only pages (admin/addBook.html, taken.html, return.html, delete.html) call the /addBook, /taken, /return and /delete routes to manage the catalogue — adding new titles, marking copies as borrowed or returned, and removing books — while /dashboard returns aggregate counts (total users, books and orders) for a simple admin overview.

7. Results and Discussion

7.1 Features Delivered

- User signup, signin and logout.
- Searchable book catalogue with category browsing.

- Cart and wishlist management.
- Checkout with order history and a simulated payment/transaction record.
- OTP generation and verification for sensitive actions.
- Purchase-gated messaging to a book's author, screened by the spam filter.
- Public Reader Wall for posting and browsing per-book reviews.
- Admin module to add, lend, return and delete books.
- Usage dashboard (total users, books and orders).
- Responsive UI styled consistently across every page.

7.2 API Endpoint Summary

Method	Endpoint	Description
POST	/signup	Create a new user account
POST	/signin	Authenticate an existing user
GET	/books	List all books in the catalogue
POST	/addBook	(Admin) add a new book
POST	/taken	(Admin) mark a book copy as borrowed
POST	/return	(Admin) mark a borrowed copy as returned
POST	/delete	(Admin) remove a book
POST	/cart/add	Add an item to the cart
GET	/cart	List cart items
DELETE	/cart/:id	Remove a cart item
POST	/wishlist/add	Add an item to the wishlist
GET	/wishlist	List wishlist items
DELETE	/wishlist/:id	Remove a wishlist item
POST	/orders	Create an order from the cart
GET	/orders/:email	Get a user's order history
DELETE	/orders/:id	Cancel/delete an order
POST	/generateOtp	Generate and email an OTP
POST	/verifyOtp	Verify a submitted OTP
POST	/payment	Record a payment transaction
GET	/payments	List payment transactions

Method	Endpoint	Description
POST	/authorMessage	Send a spam-checked message to a book's author
GET	/author/messages/:email	Get messages received by an author
POST	/authorLogin	Author login
POST	/readerWall	Post a review to the Reader Wall
GET	/readerWall	List all Reader Wall reviews
GET	/dashboard	Get aggregate usage counts

7.3 Application Walkthrough

A new reader signs up with an email and password from the landing page and signs in from the Sign In screen. From the Shop page, they can search the catalogue, add books to their cart or wishlist, read a sample preview of selected titles, or (if they have already purchased the book) send a message to its author. The Cart page lists everything added, computes a running total, and leads into checkout and OTP verification before an order is recorded. Screenshots of the running application are included in Appendix B.

7.4 Security Measures and Current Gaps

The table below summarizes what is implemented today and where the current implementation would need to be hardened before handling real user data in production — documented transparently here for both self-review and future improvement.

Area	Current State
CORS	Enabled globally via the cors middleware
Request parsing	JSON body parsing via body-parser
Password storage	Stored as plain text on the User document — not yet hashed
Session handling	No token (e.g. JWT) issued after login; the client tracks login state locally
Secondary verification	OTP model with a 300-second expiry, generated and verified server-side
Spam / abuse filtering	Rule-based checks plus a trained ML classifier for author messages
Author message access	Gated server-side on the requester having a matching prior order

8. Conclusion & Future Work

8.1 Summary of Learning

Building BookNest provided end-to-end, practical experience across the full stack: designing ten interlinked MongoDB schemas, exposing more than 30 REST routes through Express, building a multi-page vanilla-JavaScript front end that consumes that API, and integrating a self-trained machine learning model behind an independent FastAPI microservice. The project also gave hands-on exposure to deploying and coordinating two separately hosted services in production (Netlify and Render).

8.2 Skills Acquired

Project Task	Skill Category	Specific Skill Acquired
Express API design	Technical	Route/schema structuring, middleware, REST conventions
MongoDB & Mongoose	Technical	Document schema design, per-resource data modelling
Vanilla JS front end	Technical	DOM manipulation, Fetch API, localStorage-based state
Cart / checkout flow	Technical	Multi-step, stateful e-commerce UX
OTP verification	Technical	Time-limited, single-use verification codes
ML spam classifier	Technical	Text vectorization, model training with scikit-learn
FastAPI microservice	Technical	Building and serving an independent Python API
Deployment	Technical	Static hosting (Netlify), Node hosting (Render)
Project delivery	Management, Soft Skills	Independent scoping, planning, end-to-end delivery
Report writing	Soft Skills	Documenting a technical project clearly

8.3 Limitations of the Current Approach

- User passwords are stored as plain text rather than hashed (e.g. with bcrypt).
- No token-based session mechanism (e.g. JWT) is issued; login state is not cryptographically verified on subsequent requests.
- Payments are simulated (a transaction ID and status are recorded) rather than integrated with a real payment gateway.

- Book cover images are referenced by filename/path rather than uploaded to dedicated object storage.
- The admin module has no separate authentication of its own — any client can currently call the admin routes.
- No automated test suite currently covers the API routes or front-end functions.

8.4 Future Scope

- Hash passwords with bcrypt and issue JWTs (or another session token) on successful login.
- Add dedicated authentication and role checks to the admin routes.
- Integrate a real payment gateway (e.g. Razorpay/Stripe) in place of the simulated payment record.
- Support direct image uploads (e.g. to cloud object storage) instead of static file references.
- Add automated tests (unit and integration) and a CI/CD pipeline for both the Express API and the spam-detection service.
- Retrain the spam classifier periodically on new flagged messages to improve accuracy over time.

9. Internship / Project Outcomes

9.1 Technical Skills Acquired

- Hands-on experience designing and building a REST API with Node.js and Express.
- Practical schema design and querying with MongoDB Atlas and Mongoose.
- Front-end development without a framework — direct DOM manipulation and the Fetch API.
- Applied machine learning: training a text classifier and serving it through FastAPI.
- Deployment of a multi-service application across two hosting platforms.

9.2 Soft Skills Developed

- Improved debugging skills by working across the browser, the Express API and the database layers.
- Strengthened planning and time-management by independently scoping and delivering the project.
- Enhanced technical communication through structured project documentation.

9.3 Contribution to Career Growth

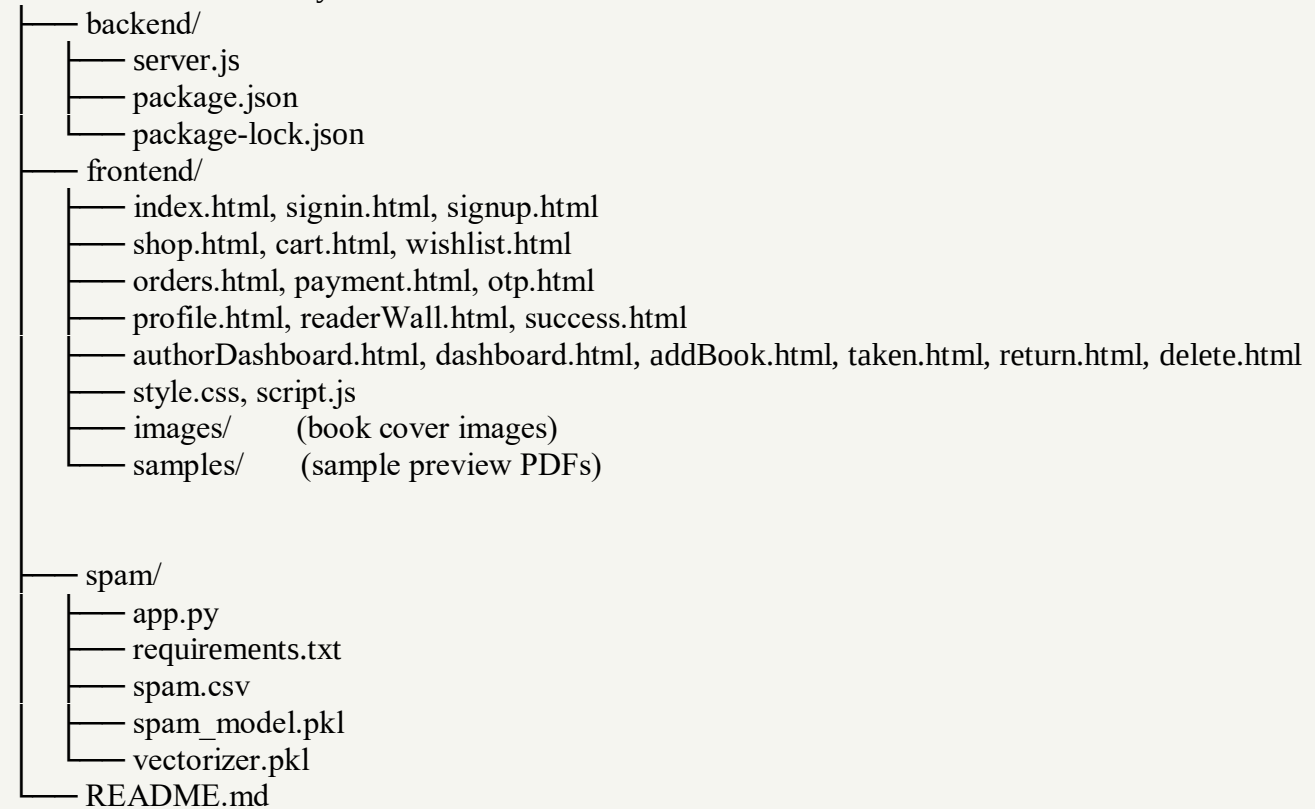
- Produced a complete, deployable full-stack project suitable for a professional portfolio.
- Gained end-to-end experience — from schema design to a publicly deployed application — relevant to full-stack and backend developer roles.
- Built a working foundation in applied machine learning that can be extended to future projects.

10. Source Code and Project Structure

10.1 Project Directory Structure

BookNest is organized as three independent modules within a single repository — an Express/MongoDB backend, a static HTML/CSS/JS frontend (with a separate admin sub-folder), and a Python/FastAPI spam-detection service.

BookNest-Smart-Library-Ecommerce-Portal/



10.2 Backend Source Code

The listings below show representative excerpts from `server.js`: the `/addBook` route used by the admin module to add a new title to the catalogue.

server.js — ***/addBook*** route

```
app.post("/addBook", async (req, res) => {
  try {
    const book = new Book({
      title: req.body.title,
      author: req.body.author,
      category: req.body.category,
      description: req.body.description,
      image: req.body.image,
```

```
price:req.body.price,
availableCopies:req.body.availableCopies,
status:"available",
rating:5
});
```

10.3 Frontend Source Code

Refer to Section 6.3 for the `loadBooks()` and `addToCart()` excerpts from `script.js`, which respectively fetch and render the catalogue and manage the client-side cart.

10.4 Spam Detection Service Source Code

The FastAPI service applies three fast rule-based checks before falling back to the trained scikit-learn model, so that obvious spam is rejected without incurring the cost of vectorization and inference:

spam/app.py — /predict-spam endpoint

```
# Spam Prediction
@app.post("/predict-spam")
def predict(request: MessageRequest):
    message = request.message.strip().lower()
    # Rule 1 - URL
    if re.search(r'https?:\/\/|www\.|', message):
        return {
            "success": True,
            "is_spam": True,
            "reason": "URL detected"
        }
    # Rule 2 - Phone Number
    if re.search(r'\b\d{10}\b', message):
        return {
            "success": True,
            "is_spam": True,
            "reason": "Phone number detected"
        }
    # Rule 3 - Spam Keywords
    spam_keywords = [
        "winner",
        "congratulations",
        "claim prize",
        "earn money",
        "click here",
        "free recharge",
        "lottery",
        "free gift",
        "cash prize",
        "win money"
```

```
]
if any(keyword in message for keyword in spam_keywords):
    return {
        "success": True,
        "is_spam": True,
        "reason": "Spam keyword detected"
    }
```

Appendix

A. Source Code (GitHub Repository Link)

<https://github.com/SamhavyaSubhanvitha/BookNest-Smart-Library-Ecommerce-Portal>

Live deployment links:

Frontend: <https://booknest-smart-library.netlify.app/>

Backend: <https://booknest-backend-nxnk.onrender.com/>

B. Application Screenshots

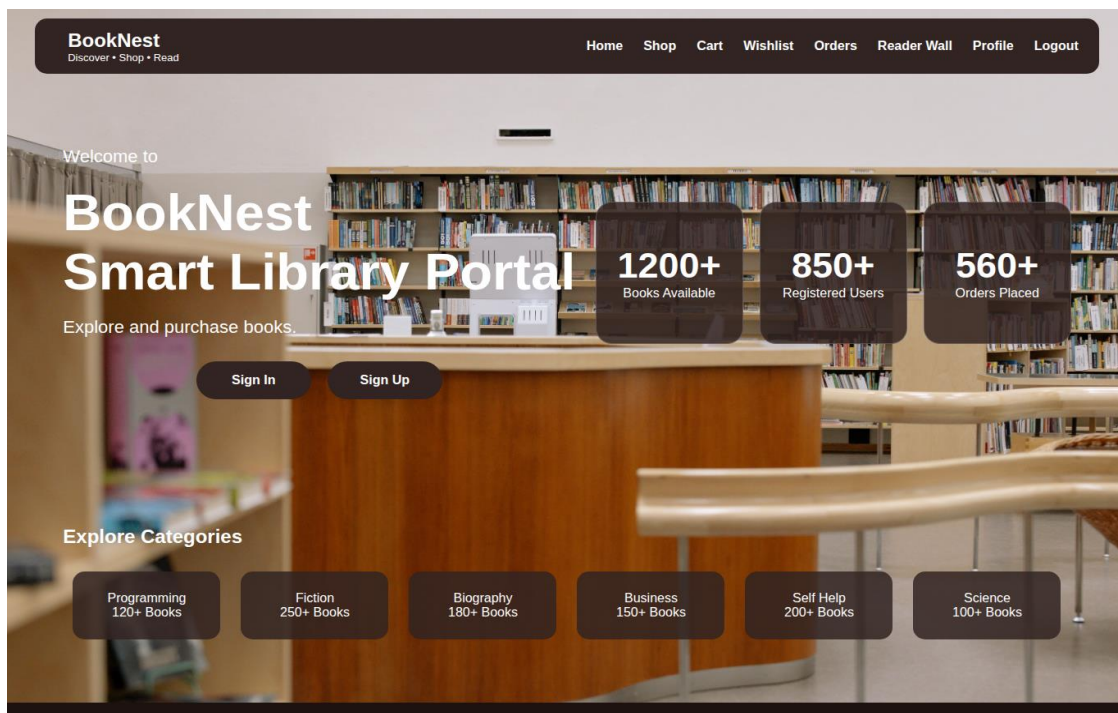


Fig. B.1 — Landing page (*index.html*)

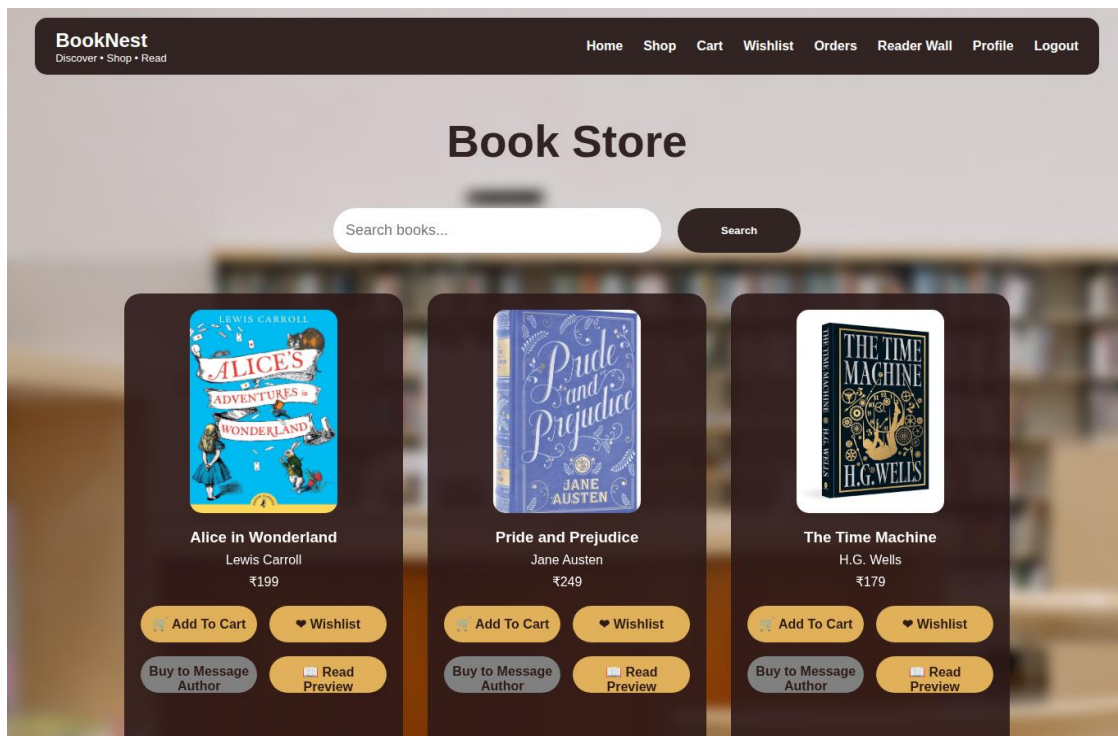


Fig. B.2 — Shop page with the book catalogue rendered from the /books API

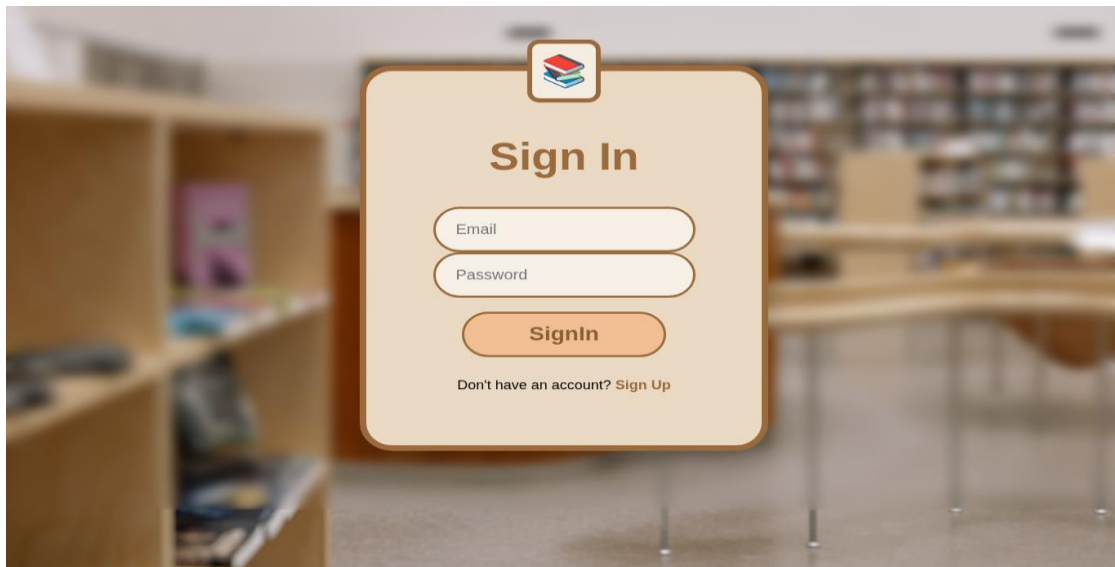


Fig. B.3 — Sign In page

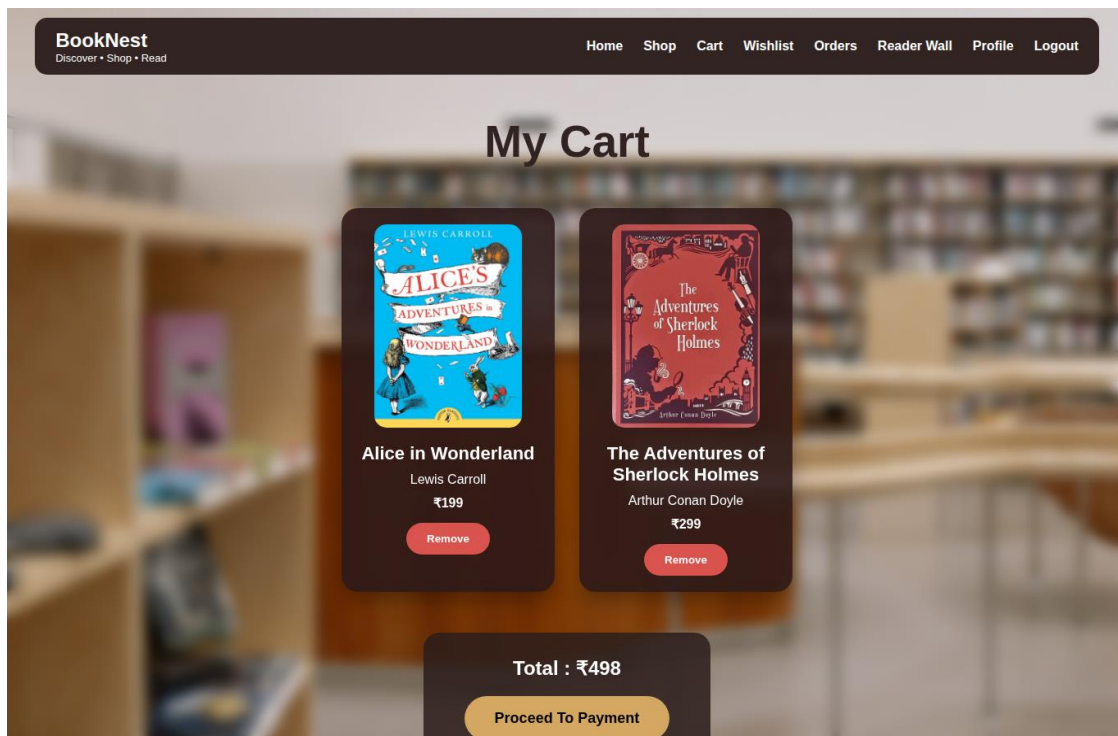


Fig. B.4 — Cart page with items added and running total

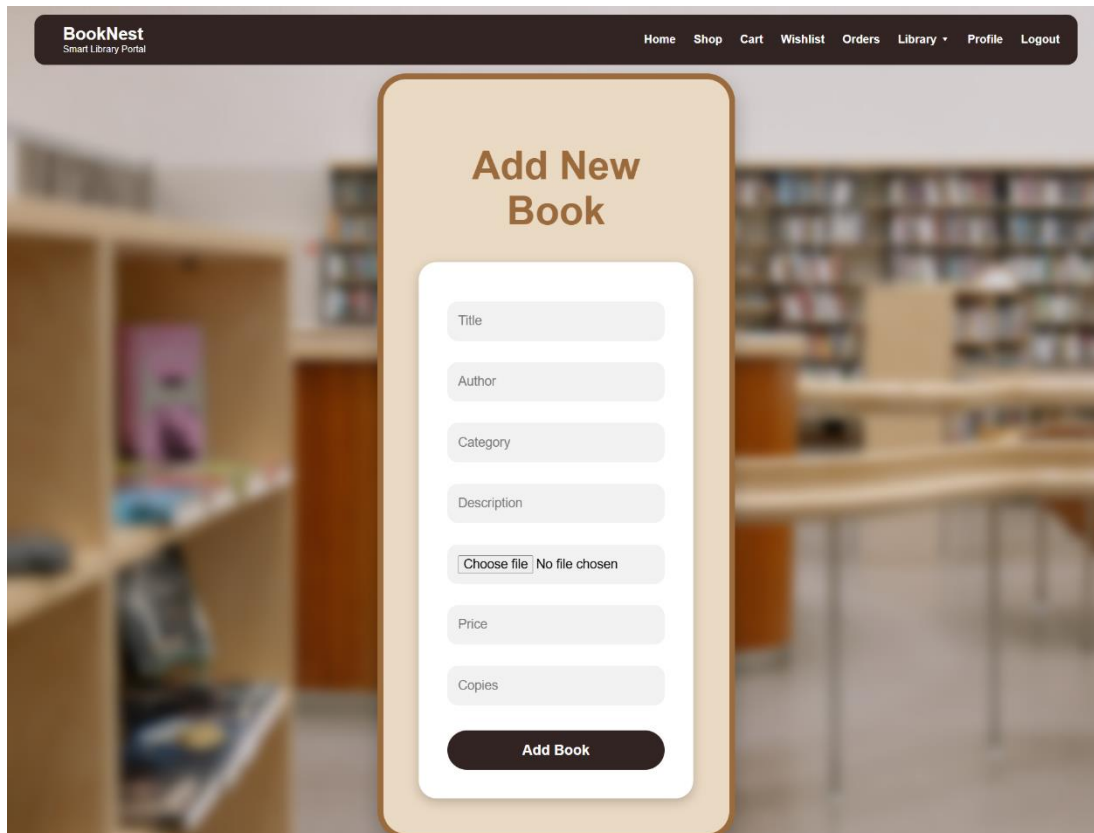


Fig. B.5 — Admin “Add Book” page

C. Certificate of Completion

